

算子优化八股文

CUDA · Roofline · GEMM · FlashAttention · Triton/
CUTLASS · Nsight

面向 CUDA/Triton/CUTLASS 与性能工程面试。所有优化都从“可复现基线 + 正确性 + profiler 假设 + 单变量实验 + 端到端收益”开始；任何单项指标都不能单独证明优化成功。

原创整理 · 公开题型检索与官方资料交叉核验 · 核验日期：2026-09-20
社区资料用于发现高频题型；技术结论以文末官方文档与原始论文为准。

建议口述顺序：先给结论，再讲机制与数据流，最后说明性能代价、适用边界和怎样用指标验证。遇到场景题，先澄清负载、硬件、SLO 与约束。

一、性能建模与访存

Q01 拿到一个慢 kernel，为什么第一步不是调 tile？

30 秒回答：先固定 GPU、驱动、shape、dtype、输入分布与计时方法，建立正确性与 p50/p95 基线，并区分 microbenchmark 和端到端收益。没有基线就无法判断新 tile 的变化来自代码、缓存、频率还是测量噪声。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q02 如何用 Roofline 决定优化方向？

30 秒回答：计算算术强度 $AI = \text{FLOPs}/\text{bytes}$ ，与硬件带宽斜线和计算平顶比较；带宽受限优先减 HBM 流量/改善访问，计算受限才优先 Tensor Core 与指令吞吐。Roofline 只是定位起点，最终要用 counter 证实实际瓶颈。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q03 什么是 global-memory coalescing？

30 秒回答：同一 warp 的 lanes 连续、对齐访问时可合并成较少内存事务；stride/gather 会浪费传输字节。优化要结合实际 tensor layout、向量化 load、边界 mask 和对齐，而不是只看源码中是否出现连续下标。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q04 什么是 shared-memory bank conflict，如何消除？

30 秒回答：同一事务访问同一 bank 的不同地址会序列化；可通过 padding、layout 重排/swizzle 或改 warp mapping 缓解。多个线程读取同一地址是 broadcast，不应误判为冲突；具体 bank 行为以目标架构文档为准。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q05 occupancy 越高越好吗？

30 秒回答：不一定。更高 occupancy 有助隐藏延迟，但大 tile、更多寄存器和 shared memory reuse 也可能提高每个 warp 的效率。应结合 occupancy 限制项、stall、吞吐和 register/shared-memory 用量判断，不能为提高 occupancy 盲目缩小 tile。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

二、GEMM、精度与流水线

Q06 register pressure 与 spilling 如何影响性能？

30 秒回答：寄存器过多会降低并发，超预算会 spill 到 local memory，而 local memory 常落在 device memory；但强压寄存器可能增加指令或破坏复用。要看编译信息与 profiler 的 local-memory 访问，再做 tile、unroll 或 launch 参数调整。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q07 高性能 GEMM 的多级 tiling 怎么映射？

30 秒回答：CTA tile 负责 block 级并行和 shared-memory reuse，warp tile 对应 warp 级 MMA，thread/register tile 负责累加与 ILP。tile 选择同时受问题 shape、dtype、寄存器、SMEM、并发和数据布局约束，跨 GPU 架构不能照抄。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q08 如何确认 Tensor Core 真正被有效使用？

30 秒回答：确认 dtype、对齐、layout、MMA shape 和实际指令/pipe 利用，而非只因使用 FP16 API 就断言满载。还需看 load/store、epilogue 与非矩阵计算是否成为新瓶颈，且用数值误差与端到端结果共同验证。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q09 FP16/BF16/FP8 算子怎样兼顾速度和正确性？

30 秒回答：分别说明输入、累加、归约、输出使用何种精度；softmax/normalization 常需稳定化与更高精度累加。用最大/相对误差、极端分布、随机种子和端到端质量回归；容差需要与参考实现和业务目标绑定。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q10 双缓冲与 async copy 的适用条件是什么？

30 秒回答：让下一 tile 的 global→shared copy 与当前 tile 的 MMA 重叠，但必须保证 producer/consumer 同步和 buffer 生命周期。若 copy/计算一方太短，或增加的寄存器/SMEM 降低并发，流水线可能无收益甚至变慢。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

三、融合、归约与 Transformer 算子

Q11 kernel fusion 为什么常加速，何时反而变慢？

30 秒回答：融合减少中间 HBM 读写和 launch，但会增大寄存器/SMEM、降低 occupancy、引入复杂分支或失去成熟库的优化。以端到端 profile 选融合边界，不能因 kernel 数少就默认更快。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q12 如何写高效且稳定的 reduction / softmax？

30 秒回答：优先使用 warp primitive 与分层 block reduction，减少全局原子竞争；softmax 先减 row max，并维护 online max/sum 以稳定数值。必须处理 mask、全无效行、边界 shape 和误差容差。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q13 FlashAttention 的核心优化是什么？

30 秒回答：它是 IO-aware tiling：把 Q/K/V tiles 放到片上存储、在线维护 softmax 状态，避免 materialize 大的 attention score/probability 矩阵，从而显著减少 HBM 往返。它仍是 exact attention，不是靠近似减少数学定义。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q14 为什么 prefill 和 decode 通常用不同 kernel 策略？

30 秒回答：prefill 的 query 序列较长、并行度丰富；低 batch decode 每请求 query 很少，KV 读取和 launch/调度更突出。应基于 profile 决定 tile、split-K、persistent kernel 或 batching，而不是把 prefill kernel 直接套到 decode。

追问提示：补充具体负载、关键指标、失败模式，以及你会用什么 trace / profiler / 对照实验验证。

Q15 MoE 的 grouped GEMM 与 AlltoAll 分别在解决什么?

30 秒回答: router 后每个 expert 的 token 数不规则, grouped GEMM 处理一组小矩阵; EP 的 token dispatch/combine 是通信问题。定位时分别测 routing、A2A、GEMM 与负载不均, 不能把 MoE 慢都归咎于 GEMM。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

四、工具链、图与工程验证

Q16 Nsight Systems 与 Nsight Compute 分别回答什么问题?

30 秒回答: Nsight Systems 看 CPU、GPU、通信与 IO 时间线, 适合找空档、同步和端到端瓶颈; Nsight Compute 深入单 kernel 的 Roofline、访存、occupancy、warp stall 和指令。常见顺序是先 Systems 定位范围, 再 Compute 优化热点。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

Q17 如何从 Nsight Compute 的现象形成优化决策?

30 秒回答: 结合 Roofline、Memory Workload、occupancy、warp stall 和指令/pipe 构造明确假设, 例如“非合并 load 导致带宽浪费”。一次只改一个变量并复测, 不迷信单项利用率, 也不跨 GPU 直接比较 raw counter。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

Q18 CUDA Graph 何时有用, 何时不适合?

30 秒回答: 重复、拓扑稳定、kernel 很短的工作流可通过 capture/replay 摊薄 CPU launch 成本; 动态 shape、频繁控制流或不兼容算子会限制收益/可用性。Graph 不能替代内核层的访存、计算或通信优化。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

Q19 Triton、CUDA、CUTLASS 如何选择并 autotune?

30 秒回答: Triton 适合快速表达与 shape/config 搜索, CUTLASS 适合复用成熟分层 GEMM 原语, CUDA/CuTe 适合特定硬件或特殊数据流。按目标 GPU、dtype、shape bucket、正确性与回退路径 benchmark, 避免一个配置泛化所有场景。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

Q20 怎样证明一个算子优化可上线?

30 秒回答: 先有 reference 与随机/极端输入正确性测试, 再做 warmup 后的稳定统计、多个 shape/dtype/架构回归和端到端压测。报告中应给出硬件/软件版本、精度容差、p50/p95、资源变化、失败回退路径和是否影响下游质量。

追问提示: 补充具体负载、关键指标、失败模式, 以及你会用什么 trace / profiler / 对照实验验证。

@小冯别放弃

60 秒答题模板

1. 定义与目标：先说它解决哪一个资源、正确性或 SLO 问题。
2. 机制与数据流：指出数据/张量如何移动，计算和通信在哪里发生。
3. 取舍：说明换来了什么显存、带宽、计算、复杂度或尾延迟代价。
4. 验证：给出应看的指标、对照实验和最小复现方式。

@小冯别放弃

资料来源与使用边界

以下社区面经/题库用于归纳问题主题；答案为原创整理。框架和硬件会演进，实际配置与行为应以对应版本的官方文档、源码和基准测试为准。

- 牛客: C++/CUDA/AI Infra 面试经验总结
- 牛客: 2025 春招 AI Infra 面经汇总
- GitHub: AI Infra 核心题清单
- GitHub: AI Infra 学习与面试高频问题
- GitHub: LLM/RL Infra 面试问题整理
- NVIDIA CUDA Programming Guide
- CUDA C++ Best Practices
- Nsight Compute Profiling Guide
- CUTLASS Efficient GEMM
- Triton Tutorials
- FlashAttention
- CUDA Graphs

@小冯别放弃